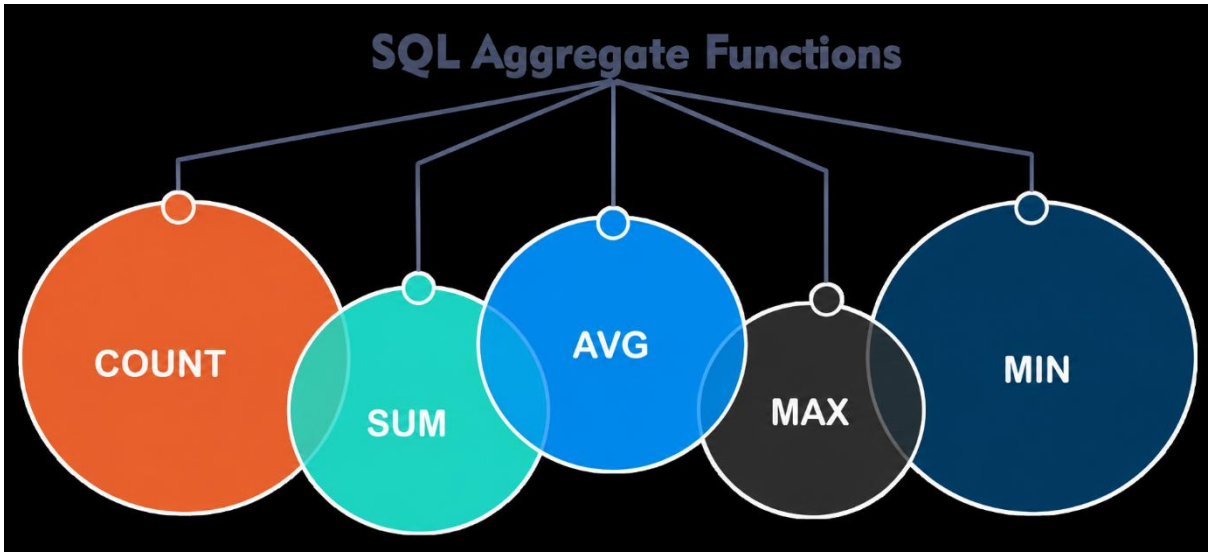
DBMS - SALES AND CUSTOMER ANALYTICS SYSTEM

Project Title: Gaming-Accessories E-Commerce Order Management System

1. Aggregate Functions

Definition

Aggregate functions are SQL functions used to perform calculations on a group of rows and return a single result.



1.1 COUNT()

Find the total number of orders:

```
SQL> SELECT COUNT(*) AS Total_Orders
  2  FROM "ORDER";

TOTAL_ORDERS
------------
           5
```

1.2 SUM()

Find the total sales amount:

```
SQL> SELECT SUM(Total_Amount) AS Total_Amount
  2  FROM "ORDER";

TOTAL_AMOUNT
------------
       11750
```

1.3 AVG()

Find the average order amount:

```
SQL> SELECT AVG(Total_Amount) AS Average_Amount
  2  FROM "ORDER";

AVERAGE_AMOUNT
--------------
          2350
```

1.4 MIN()

Find the lowest order amount:

```
SQL> SELECT MIN(Total_Amount) AS Minimum_Amount
  2  FROM "ORDER";

MINIMUM_AMOUNT
--------------
          1500
```

1.5 MAX()

Find the highest order amount:

```
SQL> SELECT MAX(Total_Amount) AS Maximum_Amount
  2  FROM "ORDER";

MAXIMUM_AMOUNT
--------------
          3200
```

2. Generate Total Sales Reports

A sales report can be generated using the SUM() function on the Total_Amount column.

```
SQL> SELECT
  2      COUNT(Order_ID) AS Total_Orders,
  3      SUM(Total_Amount) AS Total_Sales,
  4      AVG(Total_Amount) AS Average_Order_Value,
  5      MIN(Total_Amount) AS Minimum_Order,
  6      MAX(Total_Amount) AS Maximum_Order
  7  FROM "ORDER";

TOTAL_ORDERS TOTAL_SALES AVERAGE_ORDER_VALUE MINIMUM_ORDER MAXIMUM_ORDER
------------ ----------- ------------------- ------------- -------------
           5       11750                2350          1500          3200
```

The output above is based on the sample Orders data used in Gaming-Accessories

2.1 Customer-wise Sales Report

To find how much each customer has purchased:

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      SUM(o.Total_Amount) AS Total_Purchase
  5  FROM Customer01 c
  6  JOIN "ORDER" o
  7  ON c.Customer_ID = o.Customer_ID
  8  GROUP BY
  9      c.Customer_ID,
 10      c.First_Name,
 11      c.Last_Name
 12  ORDER BY Total_Purchase DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
        103
Arun Raj
          3200

        105
Karthik Mohan
          2750
```

3. Find Top Customers Based on Purchase Amount

The SUM() function can be combined with GROUP BY and ORDER BY to identify customers with higher purchase amounts.

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      SUM(o.Total_Amount) AS Total_Purchase
  5  FROM Customer01 c
  6  JOIN "ORDER" o
  7      ON c.Customer_ID = o.Customer_ID
  8  GROUP BY
  9      c.Customer_ID,
 10      c.First_Name,
 11      c.Last_Name
 12  ORDER BY Total_Purchase DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
        103
Arun Raj
          3200
```

4. Identify Best-Selling Products

The Order_Details table contains the quantity of each product ordered.

Therefore, we can use SUM(Quantity) to find the total quantity sold for each product.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(od.Quantity) AS Total_Quantity_Sold
  5  FROM Product p
  6  JOIN Order_Details od
  7      ON p.Product_ID = od.Product_ID
  8  GROUP BY
  9      p.Product_ID,
 10      p.Product_Name
 11  ORDER BY
 12      Total_Quantity_Sold DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       101
HP Laptop
                  7

       103
Samsung Galaxy
                  4
```

4.1 Product Sales Amount

Instead of only calculating quantity, we can calculate the total sales generated by each product.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(od.Quantity) AS Total_Quantity_Sold,
  5      SUM(od.Subtotal) AS Total_Sales
  6  FROM Product p
  7  JOIN Order_Details od
  8      ON p.Product_ID = od.Product_ID
  9  GROUP BY
 10      p.Product_ID,
 11      p.Product_Name
 12  ORDER BY
 13      Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
       101
HP Laptop
                  7        4700

       103
Samsung Galaxy
                  4        4050
```

5. Category-wise Sales Analysis

Category-wise analysis helps determine the sales generated by each product category.

```
SQL> SELECT
  2      c.Category_ID,
  3      c.Category_Name,
  4      SUM(od.Quantity) AS Total_Quantity_Sold,
  5      SUM(od.Subtotal) AS Total_Sales
  6  FROM Category c
  7  JOIN Product p
  8      ON c.Category_ID = p.Category_ID
  9  JOIN Order_Details od
 10      ON p.Product_ID = od.Product_ID
 11  GROUP BY
 12      c.Category_ID,
 13      c.Category_Name
 14  ORDER BY
 15      Total_Sales DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
          1
Laptops
                 11        7700

          2
Mobiles
                  4        4050
```

6. Combined Business Sales Report

A complete sales report can combine customer, order, product, and category information

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Order_Date,
  6      o.Order_Status,
  7      p.Product_ID,
  8      p.Product_Name,
  9      cat.Category_Name,
 10      od.Quantity,
 11      od.Unit_Price,
 12      od.Subtotal
 13  FROM Customer01 c
 14  JOIN "ORDER" o
 15      ON c.Customer_ID = o.Customer_ID
 16  JOIN Order_Details od
 17      ON o.Order_ID = od.Order_ID
 18  JOIN Product p
 19      ON od.Product_ID = p.Product_ID
 20  JOIN Category cat
 21      ON p.Category_ID = cat.Category_ID
 22  ORDER BY
 23      o.Order_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID ORDER_DAT ORDER_STATUS         PRODUCT_ID
---------- --------- -------------------- ----------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME
--------------------------------------------------------------------------------
  QUANTITY UNIT_PRICE   SUBTOTAL
---------- ---------- ----------
        101
```